GRAPH HACKS | TRACK 02

FounderOS

Frontend, backend and memory-layer blueprint. FalkorDB for agent memory, LangGraph for orchestration, Gemini and Groq for the LLMs, light UI for Google Stitch.

1. The idea in one paragraph

FounderOS is a team of AI agents that takes a founder from a rough idea to a launch-ready startup: validating the idea, researching the market, shaping the business model, writing the brand and pitch, and planning the first 90 days. What makes it a Track 2 entry is that the agents are not stateless chatbots. Everything they learn lives in a **FalkorDB graph that belongs to that founder**. The graph remembers across sessions, is shared between agents, records every handoff, and lets the user see *why* an agent said what it said.

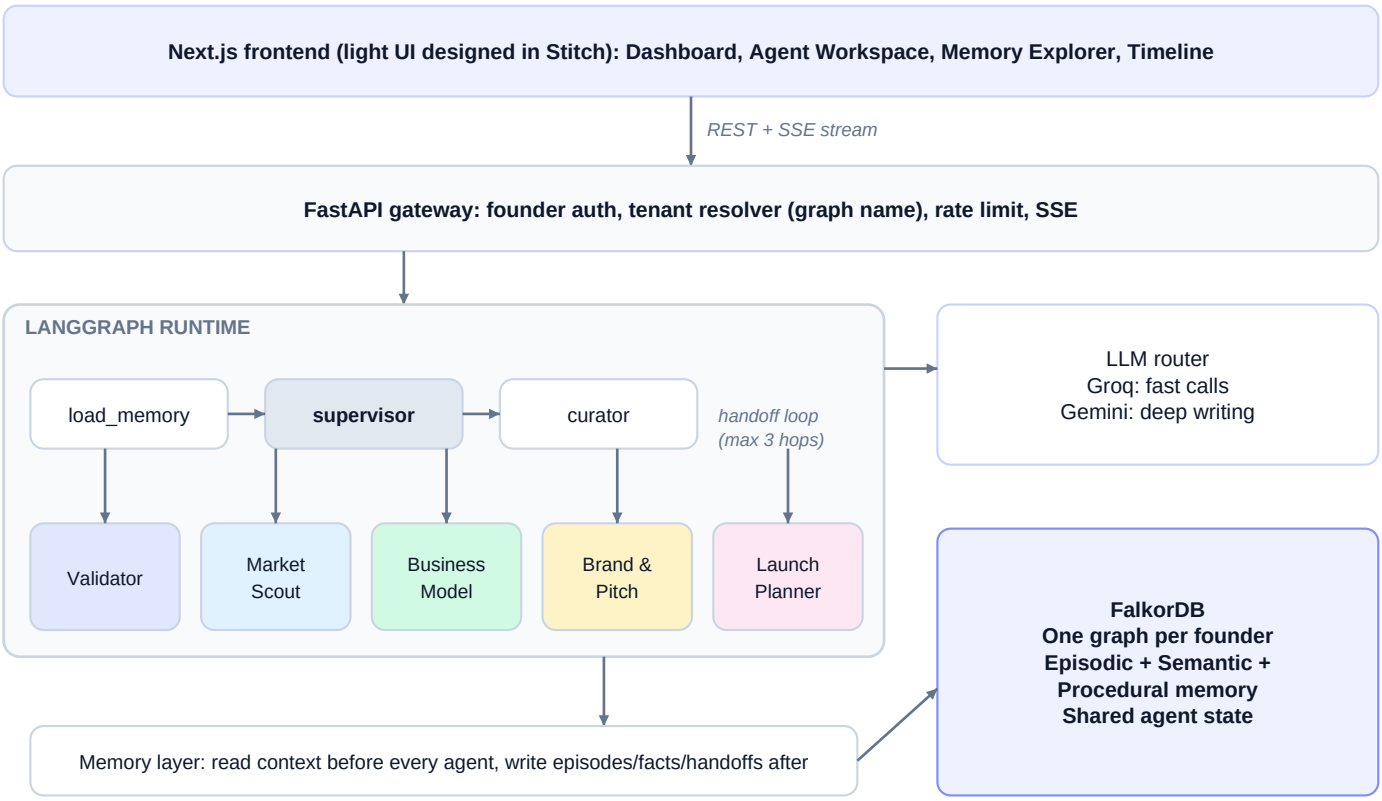
How it maps to the Track 2 brief

Track 2 requirement	How FounderOS shows it	What the judge sees in the demo
Episodic memory	Every agent turn is saved as an Episode node linked to its Session	Close the tab, reopen: agents recall last week's decisions
Semantic memory	Facts (ICP, pricing, competitors) extracted by a curator and stored as Fact nodes	Memory Explorer shows Fact nodes growing as you chat
Procedural memory	Playbook nodes (validation checklist, pitch structure) the agents follow and improve	Agent cites which playbook it used
Persistent history	Sessions and Episodes never deleted; Facts are superseded, not overwritten	Timeline view with version history
Shared state and handoffs	Agents read the same graph; handoffs are HANDOFF edges with a reason	Chip in chat: Validator hands off to Market Scout
High-concurrency reads/writes	Async FalkorDB client plus a 50-founder load script	Live counter in a Simulate page
Separate memory per tenant	One graph per founder (graph name = fos_ + founder id)	Two founders, same question, different answers

The agents

Agent	Job	LLM
Supervisor	Reads memory, decides which specialist acts next, approves handoffs	Groq (fast)
Idea Validator	Problem, target customer, risks, a validation score with reasons	Gemini
Market Scout	Competitors, market size estimate, trends, gaps	Gemini
Business Model	Lean canvas, pricing, rough unit economics	Gemini
Brand and Pitch	Name ideas, tagline, pitch outline, one-pager	Gemini
Launch Planner	30/60/90 day roadmap, legal and setup checklist, tasks	Groq (structured JSON)
Memory Curator	After each turn, extracts facts and decisions and writes them to the graph	Groq (structured JSON)

2. System architecture



Request flow. The browser sends a message to the gateway. The gateway resolves the founder's graph, starts the LangGraph run and streams events back over SSE: tokens, *agent_changed*, *memory_read* and *memory_written*. The last two events are what power the right-hand Memory panel in the UI, which is your strongest demo moment.

Suggested stack

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript + Tailwind + shadcn/ui	Fast to build, Stitch output maps cleanly to Tailwind
Graph viz	react-force-graph-2d (or Cytoscape.js)	Renders FalkorDB nodes and edges in a few lines
Client state	TanStack Query + Zustand	Server data cached, small UI store for the active agent
Backend	Python 3.11 + FastAPI + Uvicorn	Native async, easy SSE, works with LangGraph
Orchestration	LangGraph (StateGraph) + langchain-groq + langchain-google-genai	Conditional routing, handoffs, checkpointing
Memory	FalkorDB (Docker locally; FalkorDB Cloud or a small VM for the demo)	Graph per tenant, fast traversals, Cypher
Hosting	Vercel (frontend), Render or Railway (backend)	Live deployment is preferred by the hackathon

Repo layout

```
founderos/
  frontend/   (Next.js app)
  backend/
    app/main.py              FastAPI app, CORS, routers
    app/config.py            env vars: GOOGLE_API_KEY, GROQ_API_KEY, FALKOR_HOST, FALKOR_PORT
    app/llm/router.py        get_llm(task) -> Groq or Gemini, with fallbacks
    app/graph/state.py       FounderState (TypedDict)
    app/graph/builder.py     builds and compiles the LangGraph
    app/graph/nodes/         load_memory, supervisor, validator, market, model, brand, planner, curator
    app/memory/falkor.py     client, per-founder graph, schema bootstrap, Cypher helpers
    app/memory/seed.py       playbooks and demo data
    app/api/                 chat.py, memory.py, artifacts.py, founders.py
    scripts/load_test.py     simulate N founders in parallel
  docker-compose.yml         falkordb + backend
```

3. Frontend: what it must look like

Design goal: a calm, light workspace where the **memory is visible**. Most AI products hide what the model remembers. FounderOS puts it on screen next to every answer.

Screens

Screen	Layout and content	Must-have details
1. Landing	Hero, three value points, one CTA: Start building	Short and light. Do not spend more than 20 minutes here
2. Onboarding	3 steps: your name, idea in one line, current stage	On submit, backend creates the founder and their graph, then redirects
3. Dashboard	Startup header, 5-stage progress (Validate, Research, Model, Brand, Launch), next best actions, recent memory activity, open tasks	Stage progress is computed from Facts and Artifacts in the graph
4. Agent Workspace (hero screen)	Left: agent list with coloured dots. Centre: streaming chat. Right: Memory panel with two sections, **Memory used** and **Memory written**	Each message has an agent badge. Handoffs appear as a chip with the reason
5. Memory Explorer	Force-directed graph, filter chips per node type, time slider, node drawer showing provenance	Colours match the node-type tokens in section 6. Tabs: Episodic, Semantic, Procedural
6. Handoff Timeline	Vertical timeline of episodes with agent avatars, timestamps and handoff reasons	Click an episode to highlight it in the graph
7. Artifacts	Cards: Lean Canvas, Pitch Outline, Roadmap, One-pager, each with version history	Export as Markdown or PDF
8. Tasks	Kanban: Next 30 / 60 / 90 days, tasks linked to the decision that created them	Hover a task to see its source decision
9. Simulate (demo only)	Button: Simulate 50 founders. Live counters for writes per second and active graphs	Proves the high-concurrency and per-tenant requirements

Core components

- **AgentBadge**, **HandoffChip**, **MemoryCard** (fact with confidence, source episode, time), **StreamingMessage**, **GraphCanvas**, **NodeDrawer**, **StageProgress**, **ArtifactCard**, **TaskCard**, **SimulatePanel**.
- Streaming: use *fetch* with a ReadableStream against *POST /api/chat/stream*. Handle event types *token*, *agent_changed*, *memory_read*, *memory_written*, *done*.
- Memory panel updates live while the answer streams. When a new Fact is written, animate it into the panel (150 to 200 ms).

Backend endpoints the frontend calls

Method and path	Purpose
POST /api/founders	Create founder, create their FalkorDB graph, seed playbooks
POST /api/chat/stream	Run the LangGraph and stream SSE events
GET /api/memory/graph?founder_id=	Nodes and edges as JSON for the Memory Explorer
GET /api/memory/facts | episodes | handoffs	Lists for the tabs, timeline and Memory panel
GET /api/artifacts, POST /api/artifacts/{id}/regenerate	Documents and versions
GET /api/tasks, PATCH /api/tasks/{id}	Roadmap board
POST /api/simulate	Spawn N parallel founder sessions for the concurrency demo

4. Backend: LangGraph orchestration

One graph run per user message. Memory is loaded first, the supervisor picks a specialist, the specialist answers (and may request a handoff), the curator writes what was learned, then the run either ends or hops to the next specialist (maximum 3 hops).

Shared state

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class FounderState(TypedDict):
    founder_id: str
    startup_id: str
    thread_id: str
    messages: Annotated[list, add_messages]
    memory_context: dict      # facts, decisions, open tasks, recent handoffs
    active_agent: str         # chosen by supervisor
    handoff: dict | None      # {"to": "market_scout", "reason": "...", "payload": {...}}
    new_facts: list           # produced by the curator
    hops: int
```

Graph wiring

```
from langgraph.graph import StateGraph, START, END

AGENTS = {"validator": validator, "market_scout": market, "business_model": model,
          "brand_pitch": brand, "launch_planner": planner}

g = StateGraph(FounderState)
g.add_node("load_memory", load_memory)
g.add_node("supervisor", supervisor)
g.add_node("curator", curator)
for name, fn in AGENTS.items():
    g.add_node(name, fn)

g.add_edge(START, "load_memory")
g.add_edge("load_memory", "supervisor")
g.add_conditional_edges("supervisor", lambda s: s["active_agent"], list(AGENTS))
for name in AGENTS:
    g.add_edge(name, "curator")

def after_curator(s):
    h = s.get("handoff")
    return h["to"] if h and s["hops"] < 3 else "end"

g.add_conditional_edges("curator", after_curator, {**{n: n for n in AGENTS}, "end": END})
app = g.compile(checkpointer=checkpointer)   # thread_id = chat session
```

What each node does

Node	Reads from FalkorDB	Writes to FalkorDB
load_memory	Current (non-superseded) Facts, last Decisions, open Tasks, last 5 Episodes, last Handoffs	Nothing. Emits memory_read event
supervisor	memory_context from state	Nothing. Chooses active_agent with a structured-output call
Specialist agent	memory_context plus its Playbook	Nothing directly. Returns answer, optional handoff request, optional Artifact
curator	Existing Facts (to detect conflicts)	Episode, new Facts, SUPERSEDES edges, Decision, HANDOFF edge, Artifact, Tasks. Emits memory_written

Tip: let only the **curator** write to the graph. One writer keeps memory clean, makes handoffs auditable, and is simpler to debug under time pressure.

Gemini and Groq routing

```
from langchain_groq import ChatGroq
from langchain_google_genai import ChatGoogleGenerativeAI

fast = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)          # check current model IDs
deep = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.4)  # check current model IDs

def get_llm(task: str):
    if task in {"route", "extract", "summarize", "tasks_json"}:
        return fast.with_fallbacks([deep])     # speed first, Gemini as backup
    return deep.with_fallbacks([fast])         # quality first, Groq as backup
```

Task	Model	Reason
Supervisor routing, handoff decisions	Groq	Short outputs, latency matters on every turn
Curator: fact and decision extraction	Groq (structured output)	Runs after every turn, must be cheap and quick
Episode summaries	Groq	Compress long chats into one line for Episode.summary
Validator, Market, Model, Brand long-form	Gemini	Longer, higher-quality writing and larger context
Launch Planner task list	Groq (JSON)	Structured tasks with dates and owners
Embeddings (optional)	Gemini embedding model	Only if you add vector search to Facts

Model names change often. Confirm the exact IDs and free-tier rate limits in the Groq and Google AI Studio consoles before you start. Keep both API keys in environment variables, never in the repo.

5. FalkorDB memory model

Tenancy: one graph per founder, named *fos_{founder_id}*, inside a single FalkorDB instance. A read-only global graph *fos_playbooks* holds shared Playbook templates that get copied into a founder's graph on first use so the agent can improve its own copy.

Nodes and relationships

Memory type	Nodes	Key properties
Identity	Founder, Startup, Agent	id, name, idea, stage
Episodic	Session, Episode	Episode: id, agent, summary, ts, message_ref
Semantic	Fact, Competitor, Decision	Fact: key, kind (icp, pricing, risk, market...), text, confidence, updated_at
Procedural	Playbook	name, steps, version, success_count
Work products	Artifact, Task	Artifact: type, title, version. Task: title, status, due

```
(Founder)-[:OWNS]->(Startup)-[:HAS_SESSION]->(Session)-[:HAS_EPISODE]->(Episode)
(Episode)-[:BY]->(Agent)              (Episode)-[:LEARNED]->(Fact)-[:ABOUT]->(Startup)
(Episode)-[:HANDOFF {reason, ts}]->(Episode)         (Fact)-[:SUPERSEDES]->(Fact)
(Decision)-[:BASED_ON]->(Fact)        (Task)-[:FROM_DECISION]->(Decision)
(Artifact)-[:PRODUCED_BY]->(Episode)  (Episode)-[:USED]->(Playbook)
(Startup)-[:COMPETES_WITH]->(Competitor)
```

Python client and bootstrap

```
# pip install falkordb      docker run -p 6379:6379 -p 3000:3000 falkordb/falkordb:latest
from falkordb.asyncio import FalkorDB
from redis.asyncio import BlockingConnectionPool

pool = BlockingConnectionPool(max_connections=50, timeout=None, decode_responses=True,
                              host=FALKOR_HOST, port=FALKOR_PORT)
db = FalkorDB(connection_pool=pool)

def graph_for(founder_id: str):
    return db.select_graph(f"fos_{founder_id}")      # created on first write

async def bootstrap(g):
    await g.query("CREATE INDEX FOR (f:Fact) ON (f.key)")
    await g.query("CREATE INDEX FOR (e:Episode) ON (e.id)")
```

Port 3000 of the same container serves FalkorDB Browser, which is handy for debugging and for a quick look at the graph in your demo video.

Cypher you will actually use

```
// Curator: write an episode and what it learned (always parameterised)
MATCH (s:Session {id:$sid})
CREATE (e:Episode {id:$eid, agent:$agent, summary:$summary, ts:timestamp()})
CREATE (s)-[:HAS_EPISODE]->(e)
WITH e
MERGE (f:Fact {key:$key})
SET f.kind=$kind, f.text=$text, f.confidence=$conf, f.updated_at=timestamp()
CREATE (e)-[:LEARNED]->(f)

// load_memory: current facts only (not superseded)
MATCH (f:Fact) WHERE NOT (f)<-[:SUPERSEDES]-()
RETURN f.kind, f.key, f.text ORDER BY f.updated_at DESC LIMIT 25

// Handoff trail for the timeline
MATCH (a:Episode)-[h:HANDOFF]->(b:Episode)
RETURN a.agent AS from_agent, b.agent AS to_agent, h.reason, b.ts ORDER BY b.ts DESC LIMIT 20

// Explainability: why did we choose this price?
MATCH p=(d:Decision {id:$id})-[:BASED_ON*1..3]->(f:Fact)<-[:LEARNED]-(e:Episode)
RETURN p
```

Concurrency and tenancy notes

- Use the **async client** with a connection pool, and keep queries short and parameterised.
- **scripts/load_test.py**: spin up 50 simulated founders, each with its own graph, firing chat turns with *asyncio.gather*. Record writes per second for the demo.
- Isolation proof for the video: ask two different founders the same question and show different answers coming from different graphs.
- Optional upgrade if time remains: vector index on Fact text with Gemini embeddings. Check the FalkorDB docs for the current index syntax before using it. Graphiti also supports FalkorDB if you prefer a ready-made memory layer.

6. Styling for Google Stitch (light theme)

Paste the master prompt below into Stitch first, then generate each screen. Reuse the same tokens in Tailwind so the exported design and the code agree.

Colour tokens

Swatch	Token	Hex	Use
	background	#F8FAFC	App background
	surface	#FFFFFF	Cards, sidebar, panels
	surface-alt	#F1F5F9	Code blocks, table stripes, hover
	border	#E2E8F0	1px borders everywhere
	text	#0F172A	Primary text
	text-muted	#64748B	Secondary text, captions
	primary	#4F46E5	Buttons, links, active states
	primary-tint	#EEF2FF	Selected rows, badges, soft fills
	success	#16A34A	Saved, completed
	warning	#D97706	Needs attention
	danger	#DC2626	Errors only

Agent colours and memory-node colours

Swatch	Token	Hex	Use
	agent / supervisor	#475569	Supervisor avatar and badge
	agent / validator	#6366F1	Idea Validator
	agent / market	#0EA5E9	Market Scout
	agent / model	#10B981	Business Model
	agent / brand	#F59E0B	Brand and Pitch
	agent / planner	#EC4899	Launch Planner

Memory graph nodes: **Episode** #0EA5E9, **Fact** #6366F1, **Playbook** #10B981, **Decision** #F59E0B, **Task** #EC4899, **Artifact** #64748B. Use a tint of each colour (about 15 percent) for badge backgrounds and the full colour for text and graph nodes.

Type, shape and spacing

Property	Value
Fonts	Headings: Plus Jakarta Sans 600/700. Body: Inter 400/500. Code and IDs: JetBrains Mono
Type scale	12 / 14 / 16 / 20 / 28 / 40 px. Body 14 px, line height 1.5
Radius	Cards 12 px, inputs and buttons 8 px, badges and chips fully rounded
Shadows	Cards: 0 1px 2px rgba(15,23,42,0.06). Popovers: 0 8px 24px rgba(15,23,42,0.08). No heavy shadows
Spacing	4 px grid. Card padding 16 to 20 px. Section gaps 24 px
Layout	Left sidebar 248 px (white), top bar 56 px, content max width 1200 px. Workspace is 3 columns: 220 / flexible / 340 px
Icons	Lucide, 1.5 px stroke, 18 to 20 px
Motion	150 to 200 ms ease-out. Fade and slide for new memory items. No bouncing
Accessibility	Text contrast at least 4.5:1, visible focus ring in primary at 2 px, never colour alone for status

Master prompt for Stitch

```
Design a light-theme web app called FounderOS, an AI workspace that helps founders build a
startup with a team of memory-aware agents. Style: clean, calm, modern SaaS. Background
#F8FAFC, white cards with 1px #E2E8F0 borders, 12px radius, very soft shadows. Primary colour
indigo #4F46E5 with tint #EEF2FF. Text #0F172A, muted text #64748B. Fonts: Plus Jakarta Sans
for headings, Inter for body, JetBrains Mono for IDs. Lucide-style line icons. Left sidebar
(white, 248px) with the FounderOS logo, nav items: Dashboard, Workspace, Memory, Timeline,
Artifacts, Tasks. Each AI agent has a small coloured dot: Supervisor slate, Validator indigo,
Market Scout sky blue, Business Model green, Brand and Pitch amber, Launch Planner pink.
Keep it airy, with generous whitespace and no dark backgrounds.
```

Screen prompts (add after the master prompt)

Screen	Prompt to paste
Agent Workspace	Three-column layout. Left: list of 6 agents with coloured dots and a one-line status. Centre: chat with message bubbles, each with an agent badge, plus a small pill saying 'Handoff: Validator to Market Scout' between two messages, and an input bar at the bottom. Right, 340px: a Memory panel with two sections, 'Memory used' (3 fact cards with confidence and source) and 'Memory written' (2 new fact cards highlighted with a soft indigo tint).
Memory Explorer	A large force-directed graph canvas on a white card with coloured nodes (Episode sky blue, Fact indigo, Playbook green, Decision amber, Task pink). Top: filter chips per node type and a time slider. Right drawer: selected node details with 'Learned in episode', 'Used by decision' and a mini timeline.
Dashboard	Header with startup name and idea. A 5-step stage tracker (Validate, Research, Model, Brand, Launch) with progress rings. Below: 'Next best actions' cards, 'Recent memory activity' list, and 'Open tasks' list.
Handoff Timeline	Vertical timeline on a white card. Each item shows an agent avatar, time, one-line summary, and a small 'Handoff reason' chip. A subtle line connects the items.
Simulate	Card with a large 'Simulate 50 founders' button, live counters (active graphs, writes per second, p95 latency) and a small line chart. Light and minimal.

After Stitch generates a screen, export to code or Figma, then re-skin using the tokens above in tailwind.config (colors, borderRadius, fontFamily, boxShadow).

7. Build plan for today

Submissions close **tonight at 11:59 PM IST**. This plan keeps a buffer and has a cut list. Read the Rules page on WeMakeDevs once to confirm the exact submission fields.

Time (IST)	Goal	Done when
11:30 to 12:30	Setup: repo, FalkorDB in Docker, API keys, generate Stitch screens in parallel	A FastAPI route writes and reads a node in FalkorDB
12:30 to 3:30	Backend core: memory layer, LangGraph with supervisor, validator, curator, SSE	One chat turn creates an Episode and Facts in the graph
3:30 to 6:30	Frontend: onboarding, Workspace with Memory panel, Memory Explorer	You can watch facts appear live while chatting
6:30 to 8:30	Add Market Scout, Business Model, handoffs, Timeline; per-founder graphs; load script	Handoff chip appears and the Simulate numbers move
8:30 to 10:00	Deploy backend and frontend, seed demo data, fix bugs	Live URL works from a fresh browser
10:00 to 11:15	README, 2 to 3 minute demo video, submit	Submission confirmed on the platform

Cut list if you fall behind (cut in this order)

- Brand and Pitch agent, Tasks board, Artifacts versioning, Landing page polish.
- Never cut: per-founder graph, curator writing Facts and Episodes, the Memory panel, one visible handoff, the Memory Explorer graph.

Demo video script (about 2.5 minutes)

- Create a founder, describe an idea. Show the Memory panel filling with facts.
- Handoff: Validator passes to Market Scout with a reason chip. Reopen as the same founder: agents recall earlier decisions.
- Open Memory Explorer, click a Decision, show the path back to the Facts and Episodes that caused it.
- Switch to a second founder (different graph, different answers), then run Simulate 50 founders. Optional: write the blog post for the AirPods side prize.